

Universidad Politecnica de Chiapas

Ingenieria en Software

Validación de Seguridad de Entradas de Datos en la aplicación móvil AprendIA y su Backend

Materia: Seguridad de la Información

Profesor: MC. Jose Alonso Macias Montoya

Corte: Corte 3 / Unidad III: Seguridad en Aplicaciones Móviles

Alumnos:

Miguel Ángel Gutiérrez Gómez

Jesús Imanol Castillo Avendaño

Miguel Ángel Molina Gómez

Matrículas:

233382

233287

233371

Suchiapa, Chiapas, a 29 de julio de 2026

Índice

1. Introducción	3
2. Recursos Utilizados	3
3. Metodología	3
4. Alcance: por qué no se valida lo externo al cliente	4
4.1. Autenticidad e integridad del token (backend) — Fuera de alcance	4
4.2. Transporte cifrado (TLS/HTTPS) — depende de infraestructura	4
5. Tipos de entrada de datos y su validación (Cliente)	4
6. Librerías y frameworks: origen y uso (Cliente)	5
7. Desarrollo: apartados validados (Cliente, con código y evidencia)	6
7.1. 1. Validación del lado del cliente — Cumple	6
7.2. 3 y 8c. Validación de tipo / JSON (parseo defensivo) — Cumple	6
7.3. 4. Lógica de negocio — Cumple	7
7.4. 7 y 8d. Validación contextual / limpieza — Cumple	7
7.5. 8b. Filtrado (allowlist) — Cumple	8
7.6. 8f. Funciones/librerías seguras (serialización HTTP; ORM N/A) — Cumple . .	8
7.7. 8h. Canonicalización / path traversal — Cumple	8
7.8. 9. Uso de librerías/frameworks de validación — Cumple	8
7.9. 11. Gestión de errores y almacenamiento seguro — Cumple	9
8. Apartados No aplica (Cliente, justificados)	9
9. Apartado Parcial (Cliente)	10
10. Correcciones aplicadas (rama input-validate, Cliente)	10
11. Tabla Resumen de Cumplimiento (Cliente)	10
12. Índice de capturas requeridas (Cliente)	11
13. Backend: Validación de Seguridad de Entradas de Datos	12
13.1. Alcance y arquitectura	12
13.2. Recursos utilizados (Backend)	12
13.3. Metodología (Backend)	12
13.4. Mapa de puntos de entrada (Backend)	12
13.5. Tipos de entrada de datos y su validación (Backend)	13
13.6. Librerías y frameworks: origen y uso (Backend)	14
14. Backend: Desarrollo — apartados validados (con código y evidencia)	15
14.1. 2. Validación del lado del servidor — Cumple	15
14.2. 3. Validación de tipo — Cumple	16
14.3. 4. Lógica de negocio — Cumple	17
14.4. 8c. Validación de tipo de datos (estructuras JSON) — Cumple	17
14.5. 8f. Uso de funciones y librerías seguras (ORM) — Cumple	18
14.6. 8i. Escape de salida contextual (JSON) — Cumple	18
14.7. 9. Uso de librerías y frameworks de validación — Cumple	18
14.8. 11. Gestión de errores adecuada — Cumple	19

15.Backend: Apartados Parciales	19
15.1. 5. Patrones y reglas específicas — Parcial	19
15.2. 7. Validación contextual — Parcial	20
15.3. 8b. Filtrado (allowlist/blacklist) — Parcial	20
15.4. 8d. Limpieza de entradas (trim/normalize) — Parcial	20
15.5. 8j. Revisiones y auditorías (SAST/pentest) — Parcial	20
16.Backend: Apartados No aplica (justificados)	20
17.Backend: Hallazgos de seguridad y recomendaciones	21
18.Backend: Tabla Resumen de Cumplimiento	22
19.Tabla Consolidada: Cliente vs. Backend	23
20.Conclusiones Generales (Cliente + Backend)	24

1. Introducción

Este documento presenta la **validación de seguridad de las entradas de datos** de la aplicación educativa AprendIA, orientada a personas con discapacidad auditiva. El documento cubre dos componentes del sistema:

- El **cliente móvil** (Flutter/Dart, arquitectura limpia `data/domain/presentation`), evaluado originalmente contra el checklist de validación y sanitización de entradas.
- El **backend** (Spring Boot 3.4 / Java 21, `aprendia-backend`), que expone la API REST consumida por el cliente y por el panel administrativo, y que es responsable de toda la validación del lado del servidor que en la primera fase del proyecto había quedado documentada como *fuera de alcance*.

El rasgo distintivo del cliente es que **no usa formularios de usuario/contraseña ni campos de texto libre** para el flujo del estudiante: la autenticación del alumno es por código QR con token, y los ejercicios se resuelven por selección de letras/imágenes, cámara y micrófono. El backend, en cambio, sí expone formularios estructurados (JSON) para autenticación de administradores, autorregistro de usuarios, registro de estudiantes (con datos personales, dirección y familiares) y administración de catálogos (dependencias, metodologías, municipios, perfiles, roles, estados).

Trazabilidad. El trabajo del cliente está implementado y versionado en la rama `input-validate` del repositorio móvil y es verificable con `flutteranalyze`, `fluttertest` y los archivos citados. El trabajo del backend está implementado en el repositorio `aprendia-backend` (rama `summer`) y es verificable revisando las anotaciones de *Jakarta Bean Validation* en los DTO, el `GlobalExceptionHandler` y la configuración de `SecurityConfig`.

2. Recursos Utilizados

- **Cliente:** Flutter / Dart (SDK $\geq 3.4.4$), arquitectura limpia; `dio` (red), `flutter_secure_storage` (sesión cifrada); análisis estático con `flutter_lints`; pruebas con `flutter_test`; CI con GitHub Actions.
- **Backend:** Spring Boot 3.4.0 / Java 21, Maven; `spring-boot-starter-validation` (Jakarta Bean Validation / Hibernate Validator); `spring-boot-starter-security` (JWT + roles); `spring-boot-starter-data-jpa` + PostgreSQL (consultas parametrizadas); `springdoc-openapi` (documentación generada desde las mismas anotaciones de validación); Flyway (migraciones versionadas).
- **Estándar de referencia:** OWASP *MASVS* y *Mobile Top 10* (cliente); OWASP *ASVS* / *API Security Top 10* (backend).

3. Metodología

Se aplicó la misma metodología en ambos componentes: (1) **mapeo de puntos de entrada** recorriendo el código fuente (`lib/` en el cliente, `src/main/java` en el backend); (2) **evaluación apartado por apartado** del checklist institucional, con estado (**Cumple**/**Parcial**/No aplica/**Fuera de alcance**) y evidencia en código; (3) **verificación** —en el cliente con `flutteranalyze` y `fluttertest`; en el backend por inspección directa de las anotaciones de validación, del manejador global de excepciones y de la configuración de seguridad, dado que el proyecto aún no cuenta con una suite de pruebas automatizadas (ver Sección 15).

Criterio de estados

- **Cumple** — implementado y verificado.
- **Parcial** — implementado pero pendiente de un paso o incompleto para algunos campos.
- No aplica — no relevante para este componente (se justifica).
- **Fuera de alcance** — corresponde a otro componente del sistema (se justifica).

4. Alcance: por qué no se valida lo externo al cliente

4.1. Autenticidad e integridad del token (backend) — **Fuera de alcance**

Verificar firma, expiración, emisor y permisos exige material criptográfico y estado que solo tiene el servidor (principio *never trust the client*). El cliente aporta defensa en profundidad: valida el *formato* del QR, usa HTTPS en el login, manda el token en el cuerpo (no en la URL) y no lo registra en logs. Esta validación se documenta a detalle, con código, en la Sección 13.

4.2. Transporte cifrado (TLS/HTTPS) — **depende de infraestructura**

Los servicios de evaluación (20.245.212.209:5000/5001) exponen HTTP sin TLS: son IPs sin dominio ni certificado. Forzar https en el cliente antes de que el servidor ofrezca TLS dejaría la app sin conexión. **Riesgo residual: Alto** (MASVS-NETWORK); se escala como dependencia bloqueante a infraestructura. Este hallazgo sigue vigente y es independiente del trabajo de validación de entradas documentado en este informe.

5. Tipos de entrada de datos y su validación (Cliente)

La app tiene siete tipos de entrada de datos externos. La siguiente tabla mapea cada uno con su forma de validación, la **llamada a función** responsable y el archivo. Los bloques de código de cada validación se muestran en la Sección 7.

Tipo de entrada	Forma de validación y llamada a función	Archivo
Token de QR (texto)	Allowlist ASCII + longitud 8–4096: <code>validators.isValidQrToken(qrCode)</code>	<code>validators.dart</code>
Respuesta de red (JSON)	Parseo defensivo con casts seguros: <code>UserInfo.fromJson(json)</code>	<code>user_info_entity.dart</code>
Foto de cámara (imagen)	Foto y referencia no vacías: <code>ExerciseInputValidator.validateWritingInput(...)</code>	<code>evaluate_writing_exercise_usecase.dart</code>
Audio de micrófono	Audio y frase objetivo presentes: <code>ExerciseInputValidator.validateReadingInput(...)</code>	<code>evaluate_reading_exercise_usecase.dart</code>
Selección (índice/opción)	Rango/pertenencia al conjunto: <code>ExerciseInputValidator.validateSelectedOption(i,n)</code>	<code>exercise_input_validator.dart</code>
URL de medios (remota)	Canonicalización con <code>Uri.tryParse: sanitizeMediaUrl(url)</code>	<code>media_url.dart</code>
Sesión (token + datos)	Almacenamiento cifrado: <code>SessionManager.saveSession(...)</code>	<code>session_manager.dart</code>

Listado 1: Validación de la foto de cámara antes de evaluar (evaluate_writing_exercise_usecase.dart)

```
1 Future<WritingFeedbackEntity> call({
2   required File studentImage,
3   required String originalImageUrl,
4 }) async {
5   // Regla de negocio: no evaluar sin foto tomada ni imagen de referencia.
6   if (!ExerciseInputValidator.validateWritingInput(
7     photoPath: studentImage.path,
8     referenceImage: originalImageUrl,
9   )) {
10    throw ArgumentError(
11      'Entrada de escritura invalida: falta la foto o la imagen de referencia.');
```

6. Librerías y frameworks: origen y uso (Cliente)

Todas las dependencias provienen del repositorio oficial de paquetes de Dart/Flutter, **pub.dev**. La validación de entradas se implementa con un **módulo propio** (Validators), *sin* una librería externa de validación: decisión de diseño para tener un único punto auditable y no depender de terceros en la lógica de seguridad.

Librería sión)	(ver-	Origen	Uso en el manejo de entradas
flutter_secure_storage (9.2.2)		pub.dev, publisher JulianSteenbakker.nl	Guarda la sesión (token y datos del alumno, ya validados) en almacenamiento cifrado: Keystore (Android) / Keychain (iOS).
dio (5.8.0)		pub.dev, publisher dio.pub	Cliente HTTP. Serialización segura con FormData/Map/queryParameters (sin concatenar), ErrorInterceptor y LogInterceptor gateado sin cuerpos.
mobile_scanner (5.0.1)		pub.dev, publisher JulianSteenbakker.nl	Lee el valor crudo del QR (entrada externa) que después valida Validators.isValidQrToken.
equatable (2.0.5)		pub.dev, publisher felangel.dev	Igualdad por valor en las entidades de dominio (p. ej. UserInfo).
flutter_lints (4.0.0, dev)		pub.dev, equipo Dart/Flutter	Reglas de análisis estático; gate de calidad en CI.

Dependencia retirada. string_validator (1.2.0) estaba declarada pero solo se usaba de forma cosmética (isNumeric para un tamaño de fuente). Se retiró y su uso se reemplazó por Validators.isNumeric, eliminando una dependencia innecesaria de la superficie de ataque.

Herramientas de gobernanza (CI, no son dependencias de la app). osv-scanner (Google) escanea dependencias vulnerables contra la base OSV; gitleaks detecta secretos; ambas corren en GitHub Actions (.github/workflows/security.yml).

7. Desarrollo: apartados validados (Cliente, con código y evidencia)

7.1. 1. Validación del lado del cliente — **Cumple**

Estado: **Cumple.** La única entrada externa arbitraria (el token del QR) se valida por **allow-list** (ASCII imprimible, longitud 8–4096) *antes* de enviarse. No valida autenticidad (eso es del backend): es defensa en profundidad.

Dónde se cumple:

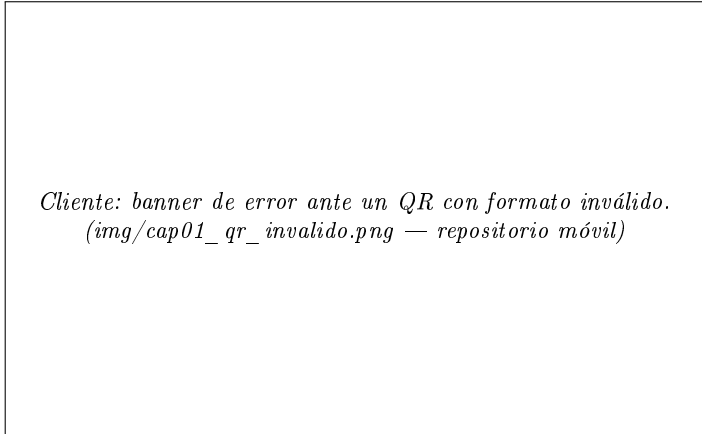
- `lib/core/validation/validators.dart`
- `lib/auth/presentation/viewmodels/qr_scanner_viewmodel.dart`

Listado 2: Validación de formato del token de QR (`validators.dart`)

```
1 static final RegExp printableAscii = RegExp(r'^[\x21-\x7E]+$');
2
3 static bool isAllowedFreeText(String value,
4     {int minLength = 1, int maxLength = 4096}) {
5     if (!hasLengthBetween(value, minLength, maxLength)) return false;
6     return printableAscii.hasMatch(value);
7 }
8
9 // Solo formato; la autenticidad la valida el backend.
10 static bool isValidQrToken(String value) =>
11     isAllowedFreeText(value, minLength: 8, maxLength: 4096);
```

Listado 3: Rechazo del QR con formato inválido antes de llamar a la API (`qr_scanner_viewmodel.dart`)

```
1 if (!Validators.isValidQrToken(qrCode)) {
2     _errorMessage = 'El código QR no tiene un formato válido.';
3     _lastScannedCode = null; // permite reintentar
4     notifyListeners();
5     return;
6 }
```



*Cliente: banner de error ante un QR con formato inválido.
(img/cap01_qr_invalido.png — repositorio móvil)*

Figura 1: Cliente: banner de error ante un QR con formato inválido.

7.2. 3 y 8c. Validación de tipo / JSON (parseo defensivo) — **Cumple**

Estado: **Cumple.** El parseo de la respuesta de login no confía en la forma del JSON: accede por pasos con casts seguros y lanza una **FormatException** controlada si falta un campo obligatorio, en vez de un crash por desreferencia de null. Usa `int.tryParse` para el id.

Dónde se cumple:

- `user_info_entity.dart`

Listado 4: fromJson defensivo (user_info_entity.dart)

```
1 factory UserInfo.fromJson(Map<String, dynamic> json) {
2   final data = json['data'];
3   if (data is! Map<String, dynamic>) {
4     throw const FormatException('Respuesta de login sin "data" valido.');
```

7.3. 4. Lógica de negocio — **Cumple**

Estado: Cumple. ExerciseInputValidator comprueba las invariantes de negocio *antes* de evaluar: audio y frase para lectura; foto e imagen de referencia para escritura; índice de opción dentro del conjunto; tema no vacío. Se aplica en los use cases de evaluación.

Dónde se cumple:

- exercises/domain/validation/exercise_input_validator.dart
- evaluate_reading_exercise_usecase.dart

Listado 5: Invariantes de negocio (exercise_input_validator.dart)

```
1 static bool validateReadingInput({
2   required String audioPath, required String targetPhrase}) {
3   if (audioPath.trim().isEmpty) return false;
4   final phrase = targetPhrase.trim();
5   return phrase.isNotEmpty && phrase.length <= maxTargetPhraseLength;
6 }
7
8 static bool validateSelectedOption(int index, int optionCount) =>
9   Validators.isInRange(index, min: 0, max: optionCount - 1);
```

7.4. 7 y 8d. Validación contextual / limpieza — **Cumple**

Estado: Cumple. Normalización unificada para comparar respuestas: minúsculas, sin tildes ni diéresis, espacios colapsados, cubriendo formas Unicode precompuestas y descompuestas. Preserva la regla de negocio: la ñ **no** se degrada a n.

Dónde se cumple:

- core/text/contextual_normalizer.dart

Listado 6: Normalización de texto (contextual_normalizer.dart)

```
1 static String normalize(String input) {
2   var s = input.toLowerCase().trim();
3   s = s.replaceAll(RegExp(r'\s+'), ' ');
4   // Protege la nn antes de tocar diacriticos y la restaura al final.
5   const enyeMarker = '#ENYE#';
6   s = s.replaceAll('n~', enyeMarker);
7   _accentMap.forEach((a, base) => s = s.replaceAll(a, base)); // a->a, u->u...
8   s = s.replaceAll(_combiningMarks, ''); // formas descompuestas (NFD)
9   return s.replaceAll(enyMarker, 'n~');
10 }
```

7.5. 8b. Filtrado (allowlist) — Cumple

Estado: Cumple. Se usa `allowlist` (cerrada por defecto) en lugar de `blacklist`: una entrada libre es válida solo si encaja en `printableAscii` y en un rango de longitud. Reutilizable desde el módulo central.

Dónde se cumple:

- `validators.dart`

7.6. 8f. Funciones/librerías seguras (serialización HTTP; ORM N/A) — Cumple

Estado: Cumple. Los paths interpolados usan `Uri.encodeComponent` y los cuerpos usan `FormData/Map`; no hay concatenación manual de URLs. Se blindo con una **prueba de arquitectura** que falla si reaparecen los patrones frágiles. **ORM = No aplica:** no hay base de datos local con consultas de usuario.

Dónde se cumple:

- `test/architecture/safe_http_serialization_test.dart`

Listado 7: Prueba de arquitectura que impide regresiones (`safe_http_serialization_test.dart`)

```
1 test('no queda ningun split(":") de URL en lib/', () {
2   final offenders = <String>[];
3   for (final file in dartFiles) {
4     if (file.readAsStringSync().contains("split(':')")) {
5       offenders.add(file.path);
6     }
7   }
8   expect(offenders, isEmpty);
9 });
```

7.7. 8h. Canonicalización / path traversal — Cumple

Estado: Cumple. `sanitizeMediaUrl` reemplaza un saneo frágil (`split(':')`) que perdía el esquema de la URL (`https://x` → `//x`). Usa `Uri.tryParse` como reja de validez.

Dónde se cumple:

- `core/net/media_url.dart`

Listado 8: Canonicalización de URLs de medios (`media_url.dart`)

```
1 String sanitizeMediaUrl(String raw) {
2   final trimmed = raw.trim();
3   if (trimmed.isEmpty) return '';
4   // Prefijo espurio antes de ':http' -> conservar desde 'http'.
5   final marker = trimmed.indexOf(':http');
6   final candidate = marker >= 0 ? trimmed.substring(marker + 1) : trimmed;
7   if (Uri.tryParse(candidate) == null) return ''; // reja de validez
8   return candidate;
9 }
```

7.8. 9. Uso de librerías/frameworks de validación — Cumple

Estado: Cumple. Módulo `Validators` como punto único de validación. Se **retiró** la dependencia `string_validator`, que estaba declarada y solo se usaba de forma cosmética, reemplazándola por `Validators.isNumeric`.

Dónde se cumple:

- `validators.dart`
- `pubspec.yaml`

7.9. 11. Gestión de errores y almacenamiento seguro — **Cumple**

Estado: **Cumple**. Mensajes de interfaz genéricos y logs sin secretos: se eliminaron/gatearon todos los `print()`; ninguno emite token, cuerpos, cabeceras ni datos personales, y el `LogInterceptor` de Dio solo se registra en `kDebugMode` y sin cuerpos. Además, la sesión se guarda **cifrada** con `flutter_secure_storage` y `clearSession()` borra las 6 claves.

Dónde se cumple:

- `dio_client.dart`
- `session_manager.dart`

Listado 9: LogInterceptor gateado y sin cuerpos (`dio_client.dart`)

```
1 if (kDebugMode) {
2   _dio.interceptors.add(LogInterceptor(
3     request: true, requestHeader: false,
4     responseHeader: false, responseBody: false,
5   ));
6 }
```

Listado 10: Sesión en almacenamiento cifrado (`session_manager.dart`)

```
1 final FlutterSecureStorage _storage; // Keystore/Keychain
2
3 Future<void> clearSession() async {
4   for (final key in _allKeys) { // las 6 claves
5     await _storage.delete(key: key);
6   }
7 }
```

8. Apartados No aplica (Cliente, justificados)

5. Patrones (email/Luhn/contraseñas)

Estado: No aplica. **Por qué no aplica:** la app no captura correo, contraseñas ni tarjetas; autenticación por QR.

6. Validación cruzada

Estado: No aplica. **Por qué no aplica:** no hay formularios multi-campo; el caso análogo (índice dentro de opciones) lo cubre `validateSelectedOption`.

8a. Escapado HTML/JS/SQL

Estado: No aplica. **Por qué no aplica:** interfaz nativa; sin SQL de usuario expuesto.

8e. Codificación

Estado: No aplica. **Por qué no aplica:** el token va en el cuerpo; se usa `Uri.encodeComponent` en segmentos de path.

8g. Reemplazo de caracteres

Estado: No aplica. **Por qué no aplica:** no hay entrada de usuario con comillas/scripts.

8i. Escape de salida

Estado: No aplica. **Por qué no aplica:** interfaz 100 % nativa Flutter.

10. Educación/capacitación

Estado: No aplica. **Por qué no aplica:** no evidenciable desde el repo; `CONTRIBUTING.md` documenta las reglas y sirve de material de incorporación.

9. Apartado Parcial (Cliente)

8j. Revisiones y auditorías (SAST/pentest) — **Parcial**

Estado: Parcial. Se añadió un workflow de CI con `flutteranalyze` como *gate*, `fluttertest`, `osv-scanner` (dependencias) y `gitleaks` (secretos), más una guía de contribución.

Dónde se cumple:

- `.github/workflows/security.yml`
- `CONTRIBUTING.md`

Por qué es parcial: el workflow debe ejecutarse en una PR real para confirmarse en producción, y el pentest manual está pendiente de agendar.

10. Correcciones aplicadas (rama input-validate, Cliente)

Brecha	Severidad	Acción
Token en <code>SharedPreferences</code> en claro; <code>clearSession</code> incompleto	Alta	<code>flutter_secure_storage</code> ; <code>clearSession</code> borra las 6 claves
Token/datos en logs (<code>print</code> + <code>LogInterceptor</code> con cuerpos)	Alta	<code>print</code> eliminados/gateados; <code>LogInterceptor</code> solo en <code>kDebugMode</code> y sin cuerpos
QR sin validación de formato	Media	<code>Allowlist</code> por regex antes de enviarlo
<code>fromJson</code> sin null-safety	Media	Parseo defensivo con <code>FormatException</code>
Reglas de negocio sin validar	Media	<code>ExerciseInputValidator</code> en los use cases
Saneamiento de URL frágil (pierde esquema)	Baja	<code>sanitizeMediaUrl</code> ; 9 copias reemplazadas
Normalización divergente (diéresis)	Baja	<code>ContextualNormalizer</code> compartido
Dependencia cosmética <code>string_validator</code>	Baja	Retirada; módulo <code>Validators</code>

Verificación reproducible (rama `input-validate`): `flutteranalyze` → **0 errores**; `fluttertest` → **22/22** pruebas de seguridad en verde (unitarias + arquitectura), y 169 pruebas del proyecto en verde; `grep` de `print(` y `split(':')` en `lib/` → **0** en código.

11. Tabla Resumen de Cumplimiento (Cliente)

#	Apartado	Estado	Evidencia
1	Validación cliente	Cumple	<code>core/validation/validators.dart</code>
2	Validación servidor	Fuera de alcance	Backend; HTTPS login en cliente
3	Validación de tipo	Cumple	<code>user_info_entity.dart</code>
4	Lógica de negocio	Cumple	<code>exercise_input_validator.dart</code>
5	Patrones (email/Luhn)	No aplica	Login QR; sin campos sensibles
6	Validación cruzada	No aplica	Sin formularios multi-campo

#	Apartado	Estado	Evidencia
7	Validación contextual	Cumple	core/text/contextual_normalizer.dart
8a	Escapado HTML/JS/SQL	No aplica	Sin WebView/SQL de usuario
8b	Filtrado (allowlist)	Cumple	validators.dart
8c	Validación tipo JSON	Cumple	user_info_entity.dart
8d	Limpieza (trim/normalize)	Cumple	contextual_normalizer.dart
8e	Codificación	No aplica	Token en cuerpo; Uri.encodeComponent
8f	Librerías seguras / ORM N/A	Cumple	test/architecture/
8g	Reemplazo de caracteres	No aplica	Sin entrada con scripts
8h	Canonicalización	Cumple	core/net/media_url.dart
8i	Escape de salida	No aplica	Interfaz nativa
8j	Revisiones/auditorías	Parcial	.github/workflows/security.yml
9	Librerías de validación	Cumple	validators.dart; pubspec.yaml
10	Educación/capacitación	No aplica	CONTRIBUTING.md
11	Gestión de errores	Cumple	dio_client.dart; session_manager.dart

Conteo final (20 apartados, Cliente): **Cumple** = 11 (1, 3, 4, 7, 8b, 8c, 8d, 8f, 8h, 9, 11) **Parcial** = 1 (8j) **No aplica** = 7 (5, 6, 8a, 8e, 8g, 8i, 10) **Fuera de alcance** = 1 (2) **No cumple** = 0.

12. Índice de capturas requeridas (Cliente)

Cada captura referida en esta sección reside en `docs/img/` del repositorio móvil (fuera del alcance de este repositorio de backend) y se documenta aquí únicamente por trazabilidad con el informe original.

Apartado	Qué debe mostrar	Archivo
1	Banner "El código QR no tiene un formato válido"	img/cap01_qr_invalido.png
3	Login exitoso tras un QR válido	img/cap03_login_ok.png
4	Botón "Enviar" deshabilitado sin la entrada requerida	img/cap04_regla_negocio.png
7	Banco de letras sin tildes ("Deletrear mi nombre")	img/cap07_deletreo.png
8f	Consola con las pruebas en verde	img/cap08_tests_verdes.png
8h	Ejercicios con imagen renderizando bien	img/cap09_imagenes_ok.png
11	logcat en login sin token ni nombre	img/cap11_log_sin_token.png

13. Backend: Validación de Seguridad de Entradas de Datos

Esta sección documenta, apartado por apartado y con código real del repositorio `aprendia-backend` (rama `somer`), todo lo relacionado con la validación de entradas que se ejecuta **del lado del servidor** — precisamente lo que la Sección 4.1 había dejado registrado como [Fuera de alcance](#) por no ser responsabilidad del cliente.

13.1. Alcance y arquitectura

El backend es una API REST construida con **Spring Boot 3.4.0** sobre **Java 21**, organizada por *features* (`feature/auth`, `feature/user`, `feature/catalogs`, `feature/healthCheck`) más un paquete transversal `security` (filtros de autenticación) y `exception` (manejo centralizado de errores). Todas las entradas externas llegan como **JSON sobre HTTP** (cuerpos de petición, variables de ruta, parámetros de consulta y cabeceras); no hay renderizado de HTML del lado del servidor ni acceso a sistema de archivos a partir de datos de usuario.

13.2. Recursos utilizados (Backend)

- **Validación declarativa:** `spring-boot-starter-validation` (Jakarta Bean Validation, implementada por Hibernate Validator), aplicada sobre *records* de Java que representan los DTO de entrada.
- **Autenticación/autorización:** `spring-boot-starter-security` + `jjwt` (JSON Web Tokens firmados con HMAC-SHA256) + `@PreAuthorize` basado en roles.
- **Persistencia:** `spring-boot-starter-data-jpa` sobre PostgreSQL, con consultas 100 % parametrizadas (sin `@Query` nativo en todo el proyecto).
- **Manejo de errores:** `GlobalExceptionHandler` (`@RestControllerAdvice`) centraliza la traducción de excepciones de validación a respuestas HTTP consistentes.
- **Documentación generada:** `springdoc-openapi` construye el contrato OpenAPI/Swagger a partir de las mismas anotaciones de validación, evitando divergencia entre documentación y código.

13.3. Metodología (Backend)

Se recorrieron los cinco puntos de entrada de la aplicación (controladores REST, cuerpos JSON deserializados, variables de ruta, parámetros de consulta y cabeceras de autenticación), se identificó el mecanismo de validación asociado a cada uno y se contrastó contra el mismo checklist institucional usado para el cliente. A diferencia del cliente, el backend **no cuenta todavía** con una suite de pruebas automatizadas (`src/test/java` está vacío) ni con un pipeline de CI; esto se documenta explícitamente como una brecha en la Sección 15.

13.4. Mapa de puntos de entrada (Backend)

Punto de entrada	Evidencia	Tipo de dato
Login admin (cuerpo JSON)	<code>AuthController.loginAdmin / UserLoginDTO</code>	Credenciales (texto)
Login estudiante (cuerpo JSON)	<code>AuthController.loginStudent / StudentQrLoginDTO</code>	Token QR (texto)
Validación de token (cuerpo JSON)	<code>AuthController.validateToken / TokenValidationRequestDTO</code>	Token JWT (texto)

Punto de entrada	Evidencia	Tipo de dato
Autorregistro de usuario (cuerpo JSON)	<code>UserController.registerUser / RegisterRequest</code>	Usuario / correo / contraseña
Registro de estudiante (JSON anidado)	<code>UserController.registerStudent / StudentRequest</code>	Persona + dirección + familiares
Catálogos administrativos (cuerpo JSON)	<code>Dependency/Metodology/Municipality/Profile/Role/StateRequest DTO</code>	Texto / numérico / booleano
Identificadores de recurso (ruta)	<code>@PathVariableLongid, @PathVariableIntegerid</code>	Numérico
Paginación (parámetros de consulta)	<code>Pageable, @PageableDefault</code>	Numérico
Token de sesión (cabecera)	<code>Authorization:Bearer<jwt> / JwtAuthenticationFilter</code>	Token (texto)
Clave de servicio (cabecera)	<code>X-API-KEY / ApiKeyFilter</code>	Clave (texto)

13.5. Tipos de entrada de datos y su validación (Backend)

Tipo de entrada	Forma de validación y llamada a función	Archivo
Credenciales de login admin	<code>@NotBlank</code> en <code>UserLoginDTO</code> , validadas por <code>AuthenticationManager.authenticate(...)</code>	<code>UserLoginDTO.java</code> , <code>AuthServiceImpl.java</code>
Código QR de estudiante	<code>@NotBlank</code> en <code>StudentQrLoginDTO</code> ; búsqueda parametrizada <code>UserRepository.findByQrCode(qr)</code>	<code>StudentQrLoginDTO.java</code>
Token JWT recibido	Verificación de firma HMAC-SHA256 y expiración: <code>JwtUtil.validateToken(token, userDetails)</code>	<code>JwtUtil.java</code> , <code>JwtAuthenticationFilter.java</code>
Registro de usuario (usuario / correo / contraseña)	<code>@NotBlank, @Email, @Size(min,max)</code> en <code>RegisterRequest</code> + <code>@Valid</code> en el controlador	<code>RegisterRequest.java</code> , <code>UserController.java</code>
Registro de estudiante (datos anidados)	Validación en cascada <code>@Valid</code> sobre <code>PersonRequest, AddressRequest</code> y <code>List<RelativeRequest></code>	<code>StudentRequest.java</code>
Referencias a catálogos (perfil, municipio, estado)	Existencia verificada en BD: <code>ensureExists(repository,id,nombre)</code>	<code>UserServiceImpl.java</code>
Duplicados de negocio (usuario, correo, CURP)	<code>UserRepository.existsByUsername/existsByEmail, PersonRepository.existsByCurp</code>	<code>UserServiceImpl.java</code>
Identificadores de ruta (id)	Coerción de tipo de Spring MVC + <code>JsonRepository.existsById(id)</code>	Controladores REST
Cuerpo JSON mal formado / tipo incorrecto	<code>HttpMessageNotReadableException</code> → <code>GlobalExceptionHandler.handle</code> <code>HttpMessageNotReadable</code>	<code>GlobalExceptionHandler.java</code>

Tipo de entrada	Forma de validación y llamada a función	Archivo
Clave de servicio (cabecera X-API-Key)	Presencia y existencia no-obsoleta: <code>ApiKeyRepository.findByKeyHash(key)</code>	<code>ApiKeyFilter.java</code>

Listado 11: DTO de registro con validación declarativa (`RegisterRequest.java`)

```

1 public record RegisterRequest(
2
3     @NotBlank(message = "El nombre de usuario es obligatorio.")
4     @Size(min = 3, max = 50, message = "El nombre de usuario debe tener entre 3 y 50
      caracteres.")
5     String username,
6
7     @NotBlank(message = "El correo electronico es obligatorio.")
8     @Email(message = "El formato de correo electronico no es valido.")
9     @Size(max = 100, message = "El correo electronico no puede superar los 100
      caracteres.")
10    String email,
11
12    @NotBlank(message = "La contraseña es obligatoria.")
13    @Size(min = 6, max = 100, message = "La contraseña debe tener entre 6 y 100
      caracteres.")
14    String password
15 ) {}

```

Listado 12: Activación de la validación en el controlador (`UserController.java`)

```

1 @PostMapping("/register")
2 @SecurityRequirements
3 public ResponseEntity<UserResponse> registerUser(
4     @Valid @RequestBody RegisterRequest registerRequest) {
5     UserResponse response = userService.registerUser(registerRequest);
6     return new ResponseEntity<>(response, HttpStatus.CREATED);
7 }

```

13.6. Librerías y frameworks: origen y uso (Backend)

Todas las dependencias provienen del repositorio oficial **Maven Central**, resueltas por Maven a partir de `pom.xml` (padre `spring-boot-starter-parent:3.4.0`). No existe una dependencia declarada explícitamente para `hibernate-validator` ni `jakarta.validation-api`: ambas llegan de forma **transitiva** a través de `spring-boot-starter-validation`.

Librería (versión)	Origen	Uso en el manejo de entradas
<code>spring-boot-starter-validation (3.4.0)</code>	Maven Central, org.springframework.boot	Habilita <code>@Valid</code> y trae transitivamente Hibernate Validator (implementación de referencia de Jakarta Bean Validation) que ejecuta <code>@NotNull</code> / <code>@NotBlank</code> / <code>@Size</code> / <code>@Min</code> / <code>@Email</code> sobre los DTO.
<code>spring-boot-starter-security (3.4.0)</code>	Maven Central, org.springframework.security	Autenticación/autorización; <code>BCryptPasswordEncoder</code> para contraseñas; <code>@PreAuthorize("hasRole(...)")</code> para permisos por método.
<code>io.jsonwebtoken:jjwt-api/impl/jackson (0.11.5)</code>	Maven Central, io.jsonwebtoken	Emisión y verificación de firma de tokens JWT (HS256) — validación de autenticidad e integridad del lado del servidor.

Librería (versión)	Origen	Uso en el manejo de entradas
spring-boot-starter-data-jpa (3.4.0)	Maven Central, o <code>g.springframework.k.boot</code>	Acceso a datos vía Hibernate ORM con consultas derivadas y parámetros ligados (<i>prepared statements</i>), previniendo inyección SQL.
com.fasterxml.jackson (transitiva, vía spring-boot-starter-web)	Maven Central, o <code>m.fasterxml.jackson.core</code>	Serialización/deserialización JSON con tipado estricto (rechaza tipos incompatibles con <code>InvalidFormatException</code>) y escapado automático de caracteres especiales en la salida JSON.
org.postgresql:postgresql (runtime)	Maven Central, o <code>g.postgresql</code>	Driver JDBC; ejecuta las sentencias preparadas generadas por Hibernate.
org.flywaydb:flyway-core	Maven Central, o <code>g.flywaydb</code>	Migraciones de esquema versionadas (integridad estructural de los datos persistentes).
springdoc-openapi-starter-webmvc-ui (2.8.5)	Maven Central, o <code>g.springdoc</code>	Genera la documentación OpenAPI/Swagger a partir de las mismas anotaciones de validación (contrato de entrada siempre sincronizado con el código).

Decisión de diseño. No se usa ninguna librería de sanitización de HTML/JS (p. ej. OWASP Java HTML Sanitizer o ESAPI) porque el backend **no renderiza HTML**: es una API JSON pura consumida por clientes que hacen su propio *output encoding* al presentar los datos.

14. Backend: Desarrollo — apartados validados (con código y evidencia)

14.1. 2. Validación del lado del servidor — **Cumple**

Estado: **Cumple.** Las cuatro subcategorías del checklist institucional (*autenticidad, consistencia, integridad, permisos*) están cubiertas:

- **Autenticidad e integridad del token:** `JwtUtil` firma los tokens con HMAC-SHA256 y, al validarlos, `Jwts.parserBuilder().setSigningKey(...).parseClaimsJws(token)` recalcula y verifica la firma; cualquier alteración del payload o de la firma lanza `SecurityException/MalformedJwtException`, y la expiración se comprueba aparte.
- **Consistencia:** las referencias a catálogos enviadas por el cliente (perfil, municipio, estado) se verifican contra la base de datos antes de persistir (`ensureExists`).
- **Permisos:** cada endpoint mutante está anotado con `@PreAuthorize("hasRole('ADMIN')")`, evaluado por Spring Security con `@EnableMethodSecurity`.

Dónde se cumple:

- `JwtUtil.java`
- `JwtAuthenticationFilter.java`
- `SecurityConfig.java`
- `UserServiceImpl.java`

Listado 13: Verificación criptográfica de la firma y expiración del JWT (`JwtUtil.java`)

```
1 private Claims getAllClaimsFromToken(String token) {
```

```

2      return Jwts.parserBuilder()
3          .setSigningKey(getSigningKey())
4          .build()
5          .parseClaimsJws(token)    // valida firma; lanza excepcion si fue alterado
6          .getBody();
7  }
8
9  public Boolean validateToken(String token, UserDetails userDetails) {
10      try {
11          final String username = getUsernameFromToken(token);
12          return (username.equals(userDetails.getUsername()) && !isTokenExpired(token))
13              ;
14      } catch (SecurityException | MalformedJwtException e) {
15          logger.error("Firma JWT invalida: {}", e.getMessage());
16      } catch (ExpiredJwtException e) {
17          logger.error("El token JWT ha expirado: {}", e.getMessage());
18      }
19      return false;
20  }

```

Listado 14: Autorización por rol a nivel de método (UserController.java)

```

1  @GetMapping
2  @PreAuthorize("hasRole('ADMIN')")
3  public ResponseEntity<List<UserResponse>> getAllUsers() { /* ... */ }
4
5  @DeleteMapping("/{id}")
6  @PreAuthorize("hasRole('ADMIN')")
7  public ResponseEntity<Void> deleteUser(@PathVariable Long id) { /* ... */ }

```

Listado 15: Reglas de autorización por ruta (SecurityConfig.java)

```

1  .authorizeHttpRequests(auth -> auth
2      .requestMatchers("/v1/auth/admin", "/v1/auth/student").permitAll()
3      .requestMatchers("/v1/users/register").permitAll()
4      .requestMatchers("/api/actuator/health", "/v1/health").permitAll()
5      .anyRequest().authenticated()
6  );
7  http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.
8      class);

```

14.2. 3. Validación de tipo — **Cumple**

Estado: **Cumple.** Todos los cuerpos de petición se deserializan a *records* de Java con campos fuertemente tipados (String, Integer, Boolean, Long); si el JSON entrante trae un tipo incompatible (por ejemplo, una cadena donde se espera un booleano), Jackson lanza `InvalidFormatException`, capturada por `GlobalExceptionHandler` y traducida a un HTTP 400 con el nombre del campo y el tipo esperado.

Dónde se cumple:

- `GlobalExceptionHandler.java`

Listado 16: Manejo de tipo de dato incorrecto en el cuerpo JSON (GlobalExceptionHandler.java)

```

1  @ExceptionHandler({HttpMessageNotReadableException.class})
2  public ResponseEntity<ErrorResponse> handleHttpMessageNotReadable(
3      HttpMessageNotReadableException ex) {
4      String detailMsg = "El formato de los datos enviados es incorrecto o el JSON esta
5          mal formado.";
6      Throwable cause = ex.getCause();
7      if (cause instanceof InvalidFormatException ife) {
8          if (!ife.getPath().isEmpty()) {
9              String fieldName = ife.getPath().get(0).getFieldName();
10             String targetType = ife.getTargetType().getSimpleName();
11             detailMsg = "Tipo de dato incorrecto, el campo '" + fieldName
12                 + "' debe ser de tipo " + targetType + ".";
13         }
14     }
15     ErrorItem errorItem = ErrorItem.builder()

```

```

14         .status("400").code("ERR_BAD_REQUEST_FORMAT")
15         .title("Bad Request Format").detail(detailMsg).build();
16     return new ResponseEntity<>(ErrorResponse.builder().errors(List.of(errorItem)).
17         build(),
18         HttpStatus.BAD_REQUEST);
19 }

```

14.3. 4. Lógica de negocio — Cumple

Estado: Cumple. UserServiceImpl aplica las reglas de negocio del dominio antes de persistir: un estudiante no puede registrarse dos veces con el mismo usuario/correo/CURP, y toda referencia a un catálogo (perfil, municipio, estado) debe existir.

Dónde se cumple:

- UserServiceImpl.java

Listado 17: Reglas de negocio de registro de estudiante (ServiceImpl.java)

```

1 public StudentResponse registerStudent(StudentRequest request) {
2     String username = request.person().curp().toLowerCase();
3
4     if (userRepository.existsByUsername(username)) {
5         throw new BadRequestException("Student is already registered (username
6             already exists).");
7     }
8     if (userRepository.existsByEmail(request.email())) {
9         throw new BadRequestException("El correo electronico ya esta registrado.");
10    }
11    if (personRepository.existsByCurp(request.person().curp())) {
12        throw new BadRequestException("El CURP proporcionado ya esta registrado en el
13            sistema.");
14    }
15    if (request.profile() != null) {
16        ensureExists(profileRepository, request.profile().longValue(), "Perfil");
17    }
18    if (request.address() != null && request.address().municipalityId() != null) {
19        ensureExists(municipalityRepository, request.address().municipalityId().
20            longValue(), "Municipio");
21    }
22    // ...
23 }
24
25 private <T> void ensureExists(JpaRepository<T, Long> repository, Long id, String
26     entityName) {
27     if (id != null && !repository.existsById(id)) {
28         throw new ResourceNotFoundException(
29             "El " + entityName + " proporcionado (" + id + ") no existe en el
30             catalogo.");
31     }
32 }

```

14.4. 8c. Validación de tipo de datos (estructuras JSON) — Cumple

Estado: Cumple. La validación en cascada de @Valid permite que estructuras JSON anidadas (objeto person dentro de StudentRequest, con su propia lista de relatives) se validen recursivamente en un único paso declarativo, sin código manual de recorrido.

Dónde se cumple:

- StudentRequest.java

Listado 18: Validación en cascada sobre estructuras JSON anidadas (StudentRequest.java)

```

1 public record StudentRequest(
2
3     @Valid
4     @NotNull(message = "Personal data is mandatory")
5     PersonRequest person,
6

```

```

7      @Valid
8      @NotNull(message = "Address is mandatory")
9      AddressRequest address,
10
11      @Valid
12      @NotNull(message = "Relatives list is mandatory")
13      List<RelativeRequest> relatives,
14
15      @NotNull(message = "Profile is mandatory")
16      Integer profile,
17
18      @NotBlank(message = "Email is mandatory")
19      String email
20 ) {}

```

14.5. 8f. Uso de funciones y librerías seguras (ORM) — Cumple

Estado: Cumple. Ningún repositorio del proyecto usa `@Query` con SQL nativo ni concatenación de cadenas: todas las consultas son métodos derivados de `JpaRepository`, resueltos por Hibernate en sentencias preparadas con parámetros ligados. Esto elimina la clase de inyección SQL clásica en la capa de acceso a datos tal como está escrita hoy.

Dónde se cumple:

- `PersonRepository.java`
- `UserRepository.java`

Listado 19: Consultas 100 por ciento parametrizadas vía Spring Data JPA (`PersonRepository.java`)

```

1  @Repository
2  public interface PersonRepository extends JpaRepository<Person, Long> {
3      boolean existsByCurp(String curp);    // -> SELECT ... WHERE curp = ? (parametro
4      Optional<Person> findByCurp(String curp);
5  }

```

14.6. 8i. Escape de salida contextual (JSON) — Cumple

Estado: Cumple. Toda respuesta se serializa con Jackson (`com.fasterxml.jackson.databind.ObjectMapper`), que escapa automáticamente comillas, barras invertidas y caracteres de control al producir el cuerpo JSON de salida, evitando que un valor persistido con comillas o caracteres especiales rompa la estructura de la respuesta. No hay generación manual de HTML/JS en el servidor, por lo que no aplica el escapado de esos contextos (ver Sección 16).

Dónde se cumple: todos los `@RestController` del proyecto (serialización automática vía `spring-boot-starter-web` / Jackson).

14.7. 9. Uso de librerías y frameworks de validación — Cumple

Estado: Cumple. Toda la validación declarativa usa **Jakarta Bean Validation**, un estándar de la plataforma Java (JSR 380) implementado por **Hibernate Validator**, ambos mantenidos activamente y ampliamente auditados; no existe ningún validador propio ni ad-hoc en el backend (a diferencia del cliente, que sí construyó un módulo propio por no depender de librerías externas en la lógica de seguridad de una app sin servidor).

Dónde se cumple:

- `pom.xml`
- DTOs listados en la Sección 13.5

Listado 20: Dependencia que habilita Jakarta Bean Validation / Hibernate Validator (`pom.xml`)

```

1  <dependency>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-starter-validation</artifactId>
4  </dependency>

```

14.8. 11. Gestión de errores adecuada — Cumple

Estado: **Cumple**. `GlobalExceptionHandler` centraliza el manejo de excepciones y evita que el servidor filtre información sensible: los errores de validación (`MethodArgumentNotValidException`) devuelven **422** con un mensaje por campo; las violaciones de integridad de datos (`DataIntegrityViolationException`) devuelven **409** con un mensaje genérico (sin exponer la sentencia SQL ni el nombre de la restricción); y cualquier excepción no controlada cae en un manejador `ExceptionHandler.class` que registra el detalle solo en el log del servidor y responde **500** con un mensaje genérico.

Dónde se cumple:

- `GlobalExceptionHandler.java`

Listado 21: Traducción de errores de validación a HTTP 422, uno por campo (`GlobalExceptionHandler.java`)

```
1  @ExceptionHandler(MethodArgumentNotValidException.class)
2  public ResponseEntity<ErrorResponse> handleValidationErrors(
3      MethodArgumentNotValidException ex) {
4      List<ErrorItem> errorItems = ex.getBindingResult().getFieldErrors().stream()
5          .map(error -> ErrorItem.builder()
6              .status("422")
7              .code("ERR_VAL_" + error.getField().toUpperCase())
8              .title("Unprocessable Entity")
9              .detail(error.getDefaultMessage())
10             .source(ErrorSource.builder()
11                 .pointer("/data/attributes/" + error.getField()).build())
12             .build())
13         .collect(Collectors.toList());
14     return new ResponseEntity<>(ErrorResponse.builder().errors(errorItems).build(),
15         HttpStatus.UNPROCESSABLE_ENTITY);
16 }
17
18 @ExceptionHandler(Exception.class)
19 public ResponseEntity<ErrorResponse> handleGeneralException(Exception ex) {
20     logger.error("Error no controlado del sistema: ", ex); // detalle solo en el log
21     // del servidor
22     ErrorItem errorItem = ErrorItem.builder()
23         .status("500").code("ERR_INTERNAL_SERVER").title("Internal Server Error")
24         .detail("Ocurrió un error interno en el servidor. Por favor, intente mas
25             tarde.")
26         .build();
27     return new ResponseEntity<>(ErrorResponse.builder().errors(List.of(errorItem)).
28         build(),
29         HttpStatus.INTERNAL_SERVER_ERROR);
30 }
```

15. Backend: Apartados Parciales

15.1. 5. Patrones y reglas específicas — Parcial

Estado: **Parcial**. El correo electrónico se valida con `@Email` en `RegisterRequest` (`@NotBlank`, `@Email`, `@Size`), y la contraseña valida longitud mínima y máxima.

Dónde se cumple:

- `RegisterRequest.java`

Por qué es parcial: `StudentRequest.email` solo tiene `@NotBlank` (sin `@Email`), por lo que un correo mal formado puede pasar por la ruta de registro de estudiante; y la contraseña no exige `@Pattern` de mezcla de mayúsculas/números/símbolos, solo longitud (`@Size(min=6,max=100)`). No aplica Luhn de tarjetas porque el dominio no maneja pagos.

Recomendación: añadir `@Email` a `StudentRequest.email` y una `@Pattern` de complejidad mínima de contraseña.

15.2. 7. Validación contextual — Parcial

Estado: **Parcial**. `ensureExists` valida que un `municipalityId` y un `stateId` existan en sus catálogos antes de persistir.

Dónde se cumple:

- `UserServiceImpl.java`

Por qué es parcial: la verificación es *individual* por campo; no comprueba que el municipio efectivamente pertenezca al estado enviado (validación cruzada contextual entre ambos campos).

Recomendación: extender `ensureExists` o agregar una comprobación específica `municipalityBelongsToState(municipalityId, stateId)`.

15.3. 8b. Filtrado (allowlist/blacklist) — Parcial

Estado: **Parcial**. Los DTO usan Bean Validation para exigir formato en algunos campos (`@Email`).

Dónde se cumple:

- `RegisterRequest.java`

Por qué es parcial: no hay una `@Pattern` de *allowlist* de caracteres para campos de texto libre como `username`, nombre de catálogos o `descripcion` de rol (columna `TEXT` sin restricción), a diferencia del cliente que sí define un allowlist ASCII explícito (`printableAscii`).

Recomendación: agregar `@Pattern` a `username` y evaluar longitud máxima para `RoleRequestDTO.descripcion`.

15.4. 8d. Limpieza de entradas (trim/normalize) — Parcial

Estado: **Parcial**. Existe normalización puntual: `request.person().curp().toLowerCase()` construye el `username` en minúsculas.

Dónde se cumple:

- `UserServiceImpl.java`

Por qué es parcial: `@NotBlank` rechaza cadenas vacías o compuestas solo por espacios, pero **no recorta** espacios al inicio/fin de una cadena válida antes de persistirla (por ejemplo, "admin" pasa la validación y se guarda con espacios).

Recomendación: aplicar `.strip()` a los campos de texto en el servicio, o un deserializador Jackson que recorte espacios globalmente.

15.5. 8j. Revisiones y auditorías (SAST/pentest) — Parcial

Estado: **Parcial**. El código se revisó manualmente para este informe (anotaciones de validación, manejo de excepciones, configuración de seguridad).

Por qué es parcial: a diferencia del cliente (que sí tiene `.github/workflows/security.yml` con `flutteranalyze`, `osv-scanner` y `gitleaks`), el backend **no tiene todavía** pipeline de CI ni herramienta de análisis estático (no hay `spotbugs`, `checkstyle` ni `sonar` configurados en `pom.xml`), y `src/test/java` está vacío — no hay pruebas unitarias ni de integración que verifiquen automáticamente estas reglas de validación.

Recomendación: agregar un workflow de GitHub Actions equivalente al del cliente (`mvnverify`, `spotbugs`, `osv-scanner`, `gitleaks`) y una primera batería de pruebas `@WebMvcTest`/`MockMvc` que ejerciten los DTO documentados en la Sección 13.5.

16. Backend: Apartados No aplica (justificados)

1. Validación del lado del cliente

Estado: No aplica para este componente. **Por qué no aplica:** es responsabilidad exclusiva del cliente móvil, ya documentada en la Sección 7.

6. Validación cruzada (fechas/campos generales)

Estado: No aplica. **Por qué no aplica:** ningún DTO actual expone pares de campos tipo "fecha inicio/fecha fin"; el único caso de coherencia entre campos identificado (municipio/estado) se documenta como **Parcial** en la Sección 15.

8a. Escapado HTML/JS/SQL

Estado: No aplica. **Por qué no aplica:** el backend no genera HTML ni JavaScript (es una API JSON pura); el escapado SQL se resuelve estructuralmente vía JPA/Hibernate (ver 8f, **Cumple**), no por escapado manual de caracteres.

8e. Codificación de entradas (Base64/URL)

Estado: No aplica. **Por qué no aplica:** ningún endpoint recibe payloads binarios en Base64; la decodificación URL de parámetros de ruta/consulta la resuelve el propio contenedor de servlets de forma transparente.

8g. Reemplazo de caracteres

Estado: No aplica. **Por qué no aplica:** no hay generación de HTML/SQL manual que requiera sustituir comillas o etiquetas de script.

8h. Canonicalización de rutas de archivo

Estado: No aplica. **Por qué no aplica:** el backend no construye rutas de sistema de archivos a partir de datos de usuario; el único acceso a `classpath` (`FirebaseConfig`) usa un recurso fijo y no controlado por el cliente.

10. Educación y capacitación

Estado: No aplica. **Por qué no aplica:** no evidenciable desde el repositorio (igual criterio que en el documento del cliente).

17. Backend: Hallazgos de seguridad y recomendaciones

Durante la elaboración de este informe se identificaron los siguientes hallazgos, adicionales a los ya cubiertos como apartados **Parcial**. Se documentan con severidad orientativa y no implican necesariamente una corrección ya aplicada.

Hallazgo	Severidad	Detalle / recomendación
Campos sin cota de longitud alineada con la base de datos (curp, zipCode, imageUrl)	Media	<code>PersonRequest.curp</code> no tiene <code>@Size(max=18)</code> pese a que la columna es <code>VARCHAR(18)</code> ; una entrada más larga falla como <code>DataIntegrityViolationException</code> (409 genérico) en vez de un 422 claro por campo. Agregar <code>@Size</code> espejo de cada <code>@Column(length=...)</code> .

Hallazgo	Severidad	Detalle / recomendación
ApiKeyFilter inyectado pero no registrado en la cadena de seguridad	Media	SecurityConfig declara @Autowired apiKeyFilter pero nunca lo añade con <code>http.addFilterBefore(...)</code> ; el filtro igual se activa globalmente por ser un @Component/OncePerRequestFilter auto-registrado por Spring Boot. Documentar explícitamente esta doble vía de activación o integrarlo formalmente a <code>SecurityFilterChain</code> .
Clave de API no está realmente hasheada	Media	El campo <code>keyHash</code> y el método <code>findByKeyHash</code> sugieren un hash, pero se compara y almacena el valor generado (<code>UUID.randomUUID()</code> concatenado) en texto plano. Aplicar una función de hash (p. ej. SHA-256) antes de persistir y comparar.
Valores por defecto de secretos en <code>application.yml</code>	Media	<code>JWT_SECRET</code> , la contraseña de base de datos y <code>ENCRYPTION_SECRET</code> tienen valores de respaldo con apariencia real hardcodeados en el archivo versionado. Aunque están pensados para sobrescribirse por variable de entorno, se recomienda usar un gestor de secretos y no comprometer valores plausibles en el repositorio.
Sin límite de tasa (<i>rate limiting</i>) ni CAPTCHA en endpoints públicos	Baja/Media	<code>/v1/auth/admin</code> , <code>/v1/auth/student</code> y <code>/v1/users/register</code> son públicos y no tienen control de fuerza bruta ni de registro masivo. Evaluar <code>Bucket4j/resilience4j</code> o un límite a nivel de proxy.
Sin cota de tamaño de <code>Pageable.size</code> en peticiones GET	Baja	No se configuró <code>spring.data.web.pageable.max-page-size</code> ; aplica el máximo por defecto de Spring Boot (2000), lo que permite solicitar páginas grandes. Fijar un máximo explícito acorde al catálogo más grande.

18. Backend: Tabla Resumen de Cumplimiento

#	Apartado	Estado	Evidencia
1	Validación cliente	No aplica	Responsabilidad del cliente móvil
2	Validación servidor	Cumple	<code>JwtUtil.java</code> , <code>SecurityConfig.java</code>
3	Validación de tipo	Cumple	<code>GlobalExceptionHandler.java</code> (Jackson)

#	Apartado	Estado	Evidencia
4	Lógica de negocio	Cumple	UserServiceImpl.java
5	Patrones (email/contraseña)	Parcial	RegisterRequest.java vs. StudentRequest.java
6	Validación cruzada	No aplica	Sin pares fecha inicio/fin en el dominio
7	Validación contextual	Parcial	ensureExists (sin cruce municipio/estado)
8a	Escapado HTML/JS/SQL	No aplica	API JSON pura; SQL resuelto por JPA
8b	Filtrado (allowlist)	Parcial	Sin @Pattern en texto libre
8c	Validación tipo JSON	Cumple	StudentRequest.java (cascada @Valid)
8d	Limpieza (trim/normalize)	Parcial	@NotBlank no recorta espacios
8e	Codificación	No aplica	Sin payloads Base64; URL decoding del contenedor
8f	Librerías seguras / ORM	Cumple	PersonRepository.java (JPA parametrizado)
8g	Reemplazo de caracteres	No aplica	Sin generación manual de HTML/SQL
8h	Canonicalización	No aplica	Sin manejo de rutas de archivo de usuario
8i	Escape de salida	Cumple	Serialización JSON vía Jackson
8j	Revisiones/auditorías	Parcial	Sin CI ni pruebas automatizadas todavía
9	Librerías de validación	Cumple	spring-boot-starter-validation
10	Educación/capacitación	No aplica	No evidenciable desde el repositorio
11	Gestión de errores	Cumple	GlobalExceptionHandler.java

Conteo final (20 apartados, Backend): **Cumple** = 8 (2, 3, 4, 8c, 8f, 8i, 9, 11) **Parcial** = 5 (5, 7, 8b, 8d, 8j) **No aplica** = 7 (1, 6, 8a, 8e, 8g, 8h, 10) **Fuera de alcance** = 0 **No cumple** = 0.

19. Tabla Consolidada: Cliente vs. Backend

La siguiente tabla cruza el estado de cada apartado en ambos componentes, mostrando cómo el backend cierra la mayoría de los apartados que el cliente había dejado **Fuera de alcance** de su alcance.

#	Apartado	Cliente	Backend	Lectura combinada
1	Validación cliente	Cumple	No aplica	Cubierto en el cliente
2	Validación servidor	Fuera de alcance	Cumple	Cerrado por el backend (JWT + roles)
3	Validación de tipo	Cumple	Cumple	Cubierto en ambas capas
4	Lógica de negocio	Cumple	Cumple	Cubierto en ambas capas
5	Patrones específicos	No aplica	Parcial	Solo relevante en backend; falta uniformar @Email
6	Validación cruzada	No aplica	No aplica	No aplica en ninguna capa hoy

#	Apartado	Cliente	Backend	Lectura combinada
7	Validación contextual	Cumple	Parcial	Backend pendiente de cruce municipio/estado
8a	Escapado HTML/JS/SQL	No aplica	No aplica	No aplica en ninguna capa
8b	Filtrado (allowlist)	Cumple	Parcial	Backend sin @Pattern de texto libre
8c	Validación tipo JSON	Cumple	Cumple	Cubierto en ambas capas
8d	Limpieza (trim/normalize)	Cumple	Parcial	Backend no recorta espacios
8e	Codificación	No aplica	No aplica	No aplica en ninguna capa
8f	Librerías seguras / ORM	Cumple	Cumple	Cubierto en ambas capas
8g	Reemplazo de caracteres	No aplica	No aplica	No aplica en ninguna capa
8h	Canonicalización	Cumple	No aplica	Solo relevante en cliente (URLs de medios)
8i	Escape de salida	No aplica	Cumple	Solo relevante en backend (serialización JSON)
8j	Revisiones/auditorías	Parcial	Parcial	Pendiente en ambas capas (backend más atrás: sin CI)
9	Librerías de validación	Cumple	Cumple	Cubierto en ambas capas
10	Educación/capacitación	No aplica	No aplica	No evidenciable en ninguna capa
11	Gestión de errores	Cumple	Cumple	Cubierto en ambas capas

20. Conclusiones Generales (Cliente + Backend)

El cliente móvil llegó a un estado conforme: 11 apartados en **Cumple**, 1 en **Parcial** (8j) y 0 en **No cumple**, tras cerrarse las brechas de mayor severidad bajo su control (token cifrado, logs sin secretos, entrada externa validada).

El backend, evaluado por primera vez en este informe, cubre **8 apartados en Cumple**, incluyendo exactamente el que el cliente había dejado **Fuera de alcance** de su alcance (validación del lado del servidor, mediante verificación criptográfica de JWT y autorización por rol). Quedan **5 apartados en Parcial**, todos con una acción concreta y acotada: uniformar @Email entre RegisterRequest y StudentRequest, cruzar municipio/estado, añadir @Pattern de allowlist a texto libre, recortar espacios en cadenas, y — el más relevante para la madurez del proyecto — incorporar un pipeline de CI con pruebas automatizadas y análisis estático, tal como ya existe en el repositorio del cliente.

Riesgos abiertos, externos a la validación de entradas: (1) TLS/HTTPS en los servicios de evaluación (infraestructura, severidad Alta, ya documentado en la Sección 4.2); (2) la clave de API no está realmente hasheada y se transmite en texto plano vía FCM; (3) valores de secretos con apariencia real en application.yml; (4) ausencia de límite de tasa en los endpoints públicos de autenticación y autorregistro.

Recomendaciones priorizadas: (1) agregar pruebas MockMvc y un workflow de CI al backend; (2) uniformar las anotaciones de Bean Validation faltantes (@Email, @Size espejo de @Column, @Pattern de allowlist); (3) hashear la clave de API y formalizar su registro en SecurityFilterChain; (4) mover los secretos de application.yml a un gestor dedicado; (5) una vez exista TLS en los servicios de evaluación, aplicar *certificate pinning* en el cliente y forzar HTTPS.